

Aerial Plane Attack

מדריך לימוד לארכיטקטורה ולקוד

מדריך תמציתי למטרת הפרויקט, לזרימת הריצה, למשפחות ה-AI, לשמירה, למגבלות ולמסלול הקוד שבחרת ללמוד.

SimulationManager -> ITrainingParadigm -> IEvolutionEngine

מוקד הקוד

להבין מי אחראי על מה, מדוע הגבולות קיימים, וכיצד הזרימה שונה בין אבולוציה ללמידת חיזוק.

זמן	כיצד להשתמש במדריך
10 דקות	עמודים 2-5: סיפור הפרויקט והארכיטקטורה.
12-15 דקות	עמודים 6-9: מסלול הקוד שבחרת.
8-10 דקות	עמודים 10-13: שמירה, סיכונים ושאלות בוחנים.

בסיס המקורות. נעשה שימוש בשני דוחות הארכיטקטורה שסיפקת: הדוח התמציתי לקביעת סדרי עדיפויות להצגה, והדוח הארוך לעומק טכני ולסיכונים. קובץ ה-Mermaid שימש למיפוי היחסים. במקרה של הבדל, קוד ה-#C הנוכחי תחת Assets/Scripts נחשב למקור הסמכות.

הוכן לפי סביבת העבודה הנוכחית, 25 ביולי 2026

1. הפרויקט בעמוד אחד

נאום של 60 שניות

Aerial Plane Attack הוא סימולטור טיסה ולמידה ב־Unity 6. ארבעה סוגי AI - Fixed NeuroEvo, PPO, NEAT ו־SAC - שולטים באותה פיזיקת מטוס ומבצעים אותן משימות. הארכיטקטורה מפרידה בין הסימולציה המשותפת למחזור החיים של הלמידה, כדי להשוות בין האלגוריתמים בלי לשכפל את המטוס או את היעדים.

חמש העובדות החשובות

- **שני יעדי אימון ממומשים:** Max Altitude ו־Flight School. ל־Dogfight קיימים מצלמה, בחירה ונשק, אך אין יעד קרבי או חיישן קרב.
- **סביבה משותפת אחת:** חיישנים מייצרים תצפיות; בקר מייצר pitch, roll, yaw, throttle; JetPhysics מזיז את אותו מטוס; והיעד מעניק ציון ומחליט מתי הריצה מסתיימת.
- **שתי משפחות אימון:** השיטות האבולוציוניות מריצות את המוחות ואת האבולוציה בתוך Unity; PPO/SAC משתמשים ב־ML-Agents ובמאמן Python חיצוני.
- **ההפשטה המרכזית:** SimulationManager מרכיב את הריצה; Paradigm מנהל את מחזור האימון; ו־Engine מנהל אבולוציית אוכלוסייה. ל־RL אין IEvolutionEngine.
- **רציפות ניסויים:** ההגדרות והשמירות מזוהות לפי (track scene, AI type), ולכן שני מסלולי Flight School אינם דורסים זה את זה.

תשובת פתיחה טובה

"הפרויקט הוא קודם כול פלטפורמת ניסויים, ולא רק משחק טיסה. ההחלטה הארכיטקטונית החשובה היא שכל שיטות ה־AI חולקות את אותם חיישנים, פעולות, פיזיקה וחוזי יעד. SimulationManager מרכיב את הריצה, ITrainingParadigm מסתיר את ההבדל בין דורות לפרקים, ו־IEvolutionEngine מפריד בין שני האלגוריתמים האבולוציוניים."

מה באמת ממומש

ניסוח לראיין	מצב	יכולת
משימת בסיס רציפה: לצבור גובה עם מאמץ בקרה נמוך.	ממומש	Max Altitude
ניווט דרך חישוקים מסודרים עם תגמולי התקדמות ויישור.	ממומש	Flight School
אבולוציית האוכלוסייה מתרחשת בתוך Unity.	ממומש	Fixed NeuroEvo / NEAT
הלמידה מתרחשת במאמן ML-Agents חיצוני.	ממומש עם תלות חיצונית	PPO / SAC
שחקן אנושי מתחרה מול AI שמור וקפוא; אין למידה או שמירה.	ממומש ב־Flight School	אתגר מדיניות שמורה
יש תשתית משחקית, אך חסרים יעד וחיישן קרביים.	לא שלם	אימון Dogfight

2. כיצד המערכת מחולקת

אפשר לחשוב על הפרויקט כעל ארבע שכבות אחריות. שלוש שכבות הקוד שבחרת יושבות מעל סביבת טיסה משותפת. השכבות מתארות בעלות על אחריות, לא תיקיות ב-Unity.

SIMULATION MANAGER

הרכבה, בחירת מצב, אוכלוסייה, חיישנים, שמירה/טעינה, כוונן ו-snapshots

TRAINING PARADIGM

מחזור חיים אבולוציוני או מחזור תהליך/פרק של RL

ENGINE / TRAINER

Classic NeuroEvo או NEAT; לחלופין מאמן Python חיצוני עבור PPO/SAC

SHARED ENVIRONMENT

חיישנים <- בקר <- JetPhysics <- תגמול ותנאי סיום

מבחן האחריות

לא אחראי על	אחראי על	שכבה
מתמטיקת בחירה/מוטציה או נוסחאות תגמול.	הרכבה, יצירת אוכלוסייה, הפעלת חיישן, מעבר מצבים, שמירה/טעינה ברמת המנהל ונתוני UI.	SimulationManager
אווירודינמיקה או הגדרת היעד.	גבולות דור באבולוציה; מחזור המאמן, ה-Academy ו-checkpoint ב-RL; שמירה והסקה לפי המשפחה.	מימוש ITrainingParadigm
מטוסים, אובייקטי סצנה, פיזיקה או יעדים.	יצירת מוחות, בחירה/מיון למינים, מוטציה/אבולוציה, אלוף וסריאליזציה אטומה.	מימוש IEvolutionEngine
איזה אלגוריתם למידה פעיל.	תצפיות, פקדים, פיזיקה, spawn, תגמולים, התקדמות ותנאי סיום.	הסביבה המשותפת

ניואנס קריטי

השרשרת בת שלוש השכבות נכונה בדיוק עבור Fixed NeuroEvo ו-NEAT. עבור PPO/SAC השרשרת היא **SimulationManager -> RLParadigm -> ML-Agents/Python trainer**. לא נכון לומר ש-RL משתמש ב-IEvolutionEngine.

חוזים תומכים חשובים

- **Objective**: בוחר חיישן, מציב מטוסים, מחשב תגמול וכושר סופי, בודק סיום וחושף פרמטרים לכוונן.
- **Sensor**: מחזיר מערך תצפית ברוחב קבוע ואת מספר הערכים.
- **IBrain**: מדיניות שניתן לקרוא לה בתוך Unity. PPO/SAC אינם חושפים מוח כזה.
- **SimulationSnapshot**: מודל קריאה יציב שמונע מה-UI להכיר מימוש Paradigm מסוים.

3. לולאת התצפית-החלטה-פעולה

1. תצפית

12 ערכי טיסה או 19 ערכי waypoint

2. החלטה

IBrain או מדיניות ML-Agents

3. פעולה

pitch, roll, yaw, throttle

4. סימולציה

JetPhysics בכל fixed tick

5. הערכה

תגמול, התקדמות וסיום

מדוע ההשוואה משמעותית

כל ארבעת סוגי ה-AI מקבלים תצפיות מאותו חיישן פעיל, מפעילים אותם ארבעה פקדי טיסה, ונשפטים בידי אותו יעד מול אותו JetPhysics. מה שמשתנה הוא מנגנון הלמידה ומחזור החיים שלו. המנהל גם מעתיק את רוחב החיישן הפעיל אל הגדרות המודל כדי למנוע אי-התאמה במספר הקלטים.

תצפיות ופעולות

מספר/טווח	מה הוא מכיל	רכיב
12 ערכים מנורמלים	מהירות מקומית, מהירות זוויתית מקומית, וקטור קדימה ווקטור למעלה.	BasicFlightSensors
19 ערכים	כל הערכים הבסיסיים ועוד כיוון מקומי ליעד, מרחק מנורמל וכיוון החישוק.	WaypointSensors
4 פלטים רציפים	Pitch, roll, yaw, throttle. המצעת מומרת מ-[-1,1] ל-[0,1].	פעולות טיסה
פלטים 5-6	ירי והחלפת נשק; אינם מקבלים ציון מיעד ממומש.	פעולות נשק אופציונליות

היעדים

חיישן	תנאי סיום	רעיון התגמול	יעד
BasicFlight	התרסקות או תום הזמן.	תוספת גובה פחות λ כפול מאמץ הבקרה. הכושר הסופי משתמש בגובה ובמאמץ הכוללים.	Max Altitude
Waypoint	התרסקות, תום זמן, יותר מדי זמן מהחישוק האחרון, סיום המסלול, או חציית מישור החישוק מחוץ לטבעת.	התקדמות, יישור קדימה, בונוס מעבר, קנס כיוון מבט, קנס מאמץ, קנס סחיפה ובונוס זמן.	Flight School

התנהגות עדינה

Flight School מעביר ליעד הבא ומעדכן את WaypointSensors בתוך `GetStepReward()`. לכן גם inference אבולוציוני חייב לקרוא לפונקציה, אף שאין למידה מהערך המוחזר.

4. מה משתנה בין ארבעת סוגי ה-AI

היכן מתרחשת הלמידה	גבול האימון	ייצוג המדיניות	סוג AI
בתוך Unity: ClassicNeuroEvoEngine.	מחכים לכל האוכלוסייה ואז מפתחים דור חדש.	רשת צפופה בעלת מבנה שכבות קבוע.	Fixed NeuroEvo
בתוך NeatEngine + Unity: SharpNEAT.	מחכים לאוכלוסייה, מצמידים fitness ומקדמים דור.	גנומים שמפתחים משקלים וטופולוגיה ומפוענחים ל-NeatBrain.	NEAT
תהליך Python חיצוני.	מטוסים מסיימים פרקים עצמאית; המאמן מעדכן מדיניות משותפת.	מדיניות ומצב optimizer של ML-Agents.	PPO
תהליך Python חיצוני.	פרקים עצמאיים ועדכוני מאמן off-policy.	מצב actor/critic והתנהגות replay של ML-Agents.	SAC

המשפחה האבולוציונית

- **אצווה סינכרונית:** מטוס סופי מקבל fitness סופי ומושבת; האבולוציה מחכה עד ש-`aliveCount` מגיע לאפס.
- **טופולוגיה קבועה:** הטובים נשמרים באליטיזם; שאר המוחות נוצרים מבחירת טורניר ומוטציה.
- **NEAT: SharpNEAT** מטפל במינים ובאבולוציה מבנית. ציונים שליליים מוזזים כלפי מעלה משום שהספרייה דורשת fitness לא-שלילי.

משפחת למידת החיזוק

- **פרקים עצמאיים:** JetMLAgent מקבל פעולה, מפעיל אותה, מבקש תגמול מהיעד ומסיים את הפרק שלו כשמגיעים למצב סופי.
- **בעלות חיצונית:** RLParadigm כותב YAML, מפעיל mlagents-learn, מגדיר BehaviorParameters ו-DecisionRequester, ומנהל resume/כיבוי.
- **אין אובייקט IBrain:** GetChampionBrain מחזיר null עבור PPO/SAC. ה-UI יכול לראות קלטים ופעולות אחרונות, לא את משקלי המאמן.

טענת ההוגנות - והגבול שלה

מה ניתן לטעון	ממשק הסביבה משותף, ולכן ההשוואה שולטת במשתני מימוש רבים. עם זאת, דורות אבולוציוניים ופרקי RL משתמשים במשטרי דגימה ועדכון שונים; זמן קיר זהה או מספר פרקים זהה אינם בהכרח תקציב ניסויי זהה.
---------------	--

תשובת השוואה שימושית

למה לא מחלקת AI ענקית אחת? מפני שלאבולוציה מבוססת דורות ול-RL עם פרקים עצמאיים אין לולאה טבעית משותפת. הממשק מאחד רק את מחזור החיים שהאפליקציה באמת צריכה: `initialize`, `tick`, `snapshot`, `persistence`, `inference` ו-`dispose`.

5. SimulationManager: הרכבה וניתוב

משפט לזיכרון

המנהל מחליט מה קיים ולאן הקריאות נשלחות; הוא אינו מבצע את הלמידה.

רצף האתחול ב-Start()

- 1 להמיר את objectiveProvider של הסצנה ל-IObjective ולעצור אם הסצנה אינה מוגדרת נכון.
- 2 לטעון הגדרות למסלול הפעיל; אם נבחר סוג AI אחר בתפריט, להשתמש בברירות המחדל המתאימות.
- 3 ליצור את האוכלוסייה, להפעיל בדיוק את החיפוש שהיעד דורש, ולהפיק ממנו את רוחב הקלט של המודל.
- 4 ליצור שירותי כוונון לתגמולים, hyperparameters וצורת הרשת.
- 5 למפות AIType ל-Paradigm הנכון ולאתחל אותו עם האוכלוסייה, ההגדרות והיעד.
- 6 אופציונלית, לטעון ריצה שמורה או להיכנס למצב challenge שהתפריט ביקש.

Managers/SimulationManager.cs - ניתוב ה-tick המרכזי

```
private void FixedUpdate()
{
    if (inChallengeMode) { TickChallenge(); return; }
    if (inInferenceMode)
    {
        activeParadigm?.TickInference();
        return;
    }
    activeParadigm?.Tick();
}
```

Managers/SimulationManager.cs - מיפוי סוג AI למחזור חיים

```
private ITrainingParadigm CreateParadigm(AIType type)
{
    switch (type)
    {
        case AIType.FixedNeuroEvo:
            return new EvolutionaryParadigm(new ClassicNeuroEvoEngine());
        case AIType.NEAT:
            return new EvolutionaryParadigm(new NeatEngine());
        case AIType.PPO_MLAgents:
        case AIType.SAC_MLAgents:
            return new RLParadigm();
        default:
            return null;
    }
}
```

מה לציין בקוד

- זוהי הזרקת תלות דרך constructor: אותו EvolutionaryParadigm מקבל אחד משני Engines.
- PPO ו-SAC חולקים RLParadigm; סוג ה-AI משנה את ה-YAML וההגדרות.
- המנהל מפעיל בכל FixedUpdate מחזור חיים אחד בלבד: training, inference או challenge.

6. TrainingParadigm: הסתרת הבדלי מחזור החיים

זהו Strategy הפונה אל שכבת האפליקציה. המנהל יכול להתחיל, לבצע tick, לקרוא telemetry, לשמור/לטעון, להיכנס ל-inference ולנקות משאבים בלי לדעת אם האלגוריתם רץ באצוות #C או בפרקים דרך Python.

Managers/Paradigms/ITrainingParadigm.cs - המתודות שכדאי לזכור

```
void Initialize(List<JetAgent> population,
               SimulationSettings settings,
               IObjective objective);
void Tick();
SimulationSnapshot GetSnapshot();
void SaveState();
void LoadState();
bool CanRunInference { get; }
bool StartInference();
void TickInference();
void Dispose();
```

EvolutionaryParadigm מנהל דור

5. spawn מחדש

שיוך מוחות חדשים

4. אבולוציה

כאשר `aliveCount == 0`

3. הוצאה

סיום מטוסים terminal

2. הערכה

תגמול לכל מטוס חי

1. שיוך

ה-engine יוצר מוחות

- במהלך ההערכה: `CurrentFitness += GetStepReward()`. בסיום: `CalculateTotalFitness()`, השבתה והקטנת `aliveCount`.
- בגבול הדור: איסוף ציונים, קריאה ל-`EvolveNextGeneration()`, עדכון סטטיסטיקה, שיוך מוחות ו-spawn מחדש.

RLParadigm מנהל את מחזור המאמן

- Initialize אינו מפעיל Python. ה-Tick הראשון מתחיל ריצה חדשה; LoadState יכול להכין checkpoint ולהתחיל ישירות ב-resume.
- BehaviorParameters מוגדר לפני הוספה/הפעלה של JetMLAgent, כדי למנוע רישום עם ברירת מחדל של אפס פעולות.
- JetMLAgent, ולא RLParadigm.Tick, מנהל את לולאת הפעולה-תגמול-סיום. ה-Paradigm מנהל תהליך, סטטיסטיקה, persistence, Academy וניקוי.

הממשק אינו מעמיד פנים שהלולאות זהות. הוא מאחד רק את מה ששאר האפליקציה באמת צריכה. זהו Strategy ברמת מחזור החיים של האימון.

הסבר טכנוני טוב

7. EvolutionEngine: בידוד אלגוריתם האוכלוסייה

engine - גבול ה- Managers/Engines/ EvolutionEngine.cs

```
List<IEvolvableBrain> InitializeGeneration(SimulationSettings settings);
List<IEvolvableBrain> EvolveNextGeneration(List<float> fitnessScores);
int GetLastGenerationBestEliteIndex();
IEvolvableBrain GetChampionBrain();
float GetChampionScore();
string CaptureState();
List<IEvolvableBrain> RestoreState(
    string stateJson, SimulationSettings settings);
```

ClassicNeuroEvoEngine

- יוצר NeuroEvoBrain צפוף אחד לכל מקום באוכלוסייה לפי מבנה הרשת.
- ממיין זוגות מוח/ציון, מעדכן אלוף היסטורי ושומר לפחות את האחוז העליון כ- elites.
- לכל ילד שאינו elite מבצע טורניר של חמישה מועמדים, מעתיק את המנצח ומבצע מוטציה.
- קורא הורים מ- parentPool בלתי משתנה, כדי שילדים שכבר נכתבו לא יהפכו בטעות להורים.
- CaptureState שומר shape, כל המוחות השטוחים, אלוף, ציון ודור ב-JSON.

NeatEngine

- בונה את מפעל הגנומים, speciation, decoder, ואת אלגוריתם האבולוציה של SharpNEAT.
- מצמיד ציונים לאובייקטי הגנום הנוכחיים לפני StepOneGeneration, משום שהספרייה בונה ומסדרת מחדש את הרשימה.
- מזיז ציונים שליליים ומוסיף baseline כך שכל ערכי ה- fitness חיוביים.
- מגן על שני האחוזים העליונים מפני אובדן elite הסתברותי ומאתר מחדש את מקום המנצח לצורך המצלמה.
- CaptureState שומר XML מלא של האוכלוסייה ושל האלוף בתוך JSON payload אטום.

EvolutionaryParadigm אינו יודע אם המוח הוא משקלים שטוחים או גנום SharpNEAT. הוא שומר רק את מחרוזת EngineState, ופרטי הסריאליזציה נשארים ב- engine שבבעלותו.

למה מצב אטום חשוב

היכן ההפשטה אינה מושלמת

IEvolvableBrain חושף Copy ו- Mutate, דבר שמתאים לטופולוגיה קבועה אך לא ל- NEAT, שבו הפעולות שייכות לגנום/ engine. פיצול נקי יותר יפריד בין phenotype שמוגל לבצע inference לבין מוח fixed-topology שניתן לשנות ישירות.

8. שתי עקיבות מקצה לקצה

עקיבה א': דור אחד של Fixed NeuroEvo

1. SimulationManager יוצר EvolutionaryParadigm(new ClassicNeuroEvoEngine()).
2. Initialize מבקש מוחות מה-engine, משייך מוח לכל JetAgent, מגדיר cadence וקורא ל-SetStartingState.
3. JetAgent קורא מדי כמה ticks את החייושן, מעביר תצפיות ל-Brain.GetControlOutputs ומשתמש באותה פעולה בין החלטות.
4. JetPhysics משלב דחף, עילוי, גרר, מומנט, damping ויציבות בכל צעד פיזיקה.
5. EvolutionaryParadigm צובר תגמול. מטוס terminal מקבל fitness סופי ומושב.
6. כשכל המטוסים הסתיימו, ה-engine מקבל ציונים, מחזיר מוחות שהתפתחו, וה-Paradigm יוצר את הדור הבא.

עקיבה ב': פרק אחד של PPO/SAC

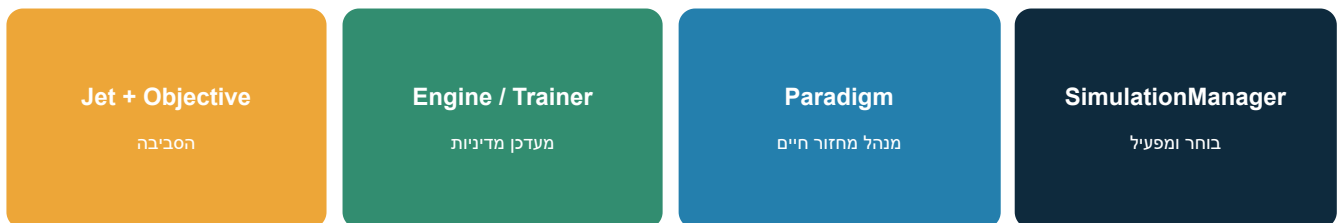
1. SimulationManager יוצר RLParadigm. ה-Tick הראשון כותב YAML ומפעיל mlagents-learn, אלא אם זה מסלול resume/inference.
2. RLParadigm מגדיר BehaviorParameters ו-DecisionRequester, ומזריק הפניות ל-JetMLAgent.
3. CollectObservations מעתיק את וקטור החייושן אל ML-Agents.
4. מדיניות Python מחזירה פעולות רציפות; JetMLAgent מפעיל אותן על אותו JetPhysics.
5. JetMLAgent מבקש תגמול מאותו Objective, קורא ל-AddReward ומסיים את הפרק במצב terminal.
6. OnEpisodeBegin מדווח את הציון הקודם, מאפס את המטוס ומבקש מצב התחלה חדש.

אם שואלים היכן מתרחשת הלמידה

Fixed/NEAT: engine C# Unity מחזיר אוכלוסייה חדשה.

PPO/SAC: Python מעדכן מדיניות משותפת מטרג'קטוריות שנשלחו דרך ML-Agents; Unity היא הסביבה.

התרשים הפשוט שכדאי לדעת לצייר



9. שמירה, inference, כונון ו-challenge

זהות השמירה וה-payload

- **זהות:** `Track.GameData/<Track>/save_<AIType>.json`. הוא שם הסצנה המסונן; mode עדיין בוחר ברירות מחדל וחיישן.
- **תוכן משותף:** עותק של `SimulationSettings`, פרמטרי יעד, מספר דור/פרק, סטטיסטיקה והיסטוריה, זמן ומצב engine אטום.

התנהגות resume	מה נשמר	מסלול
ה-engine משחזר מוחות; ה-Paradigm משייך אותם ומבצע .spawn	כל המוחות השטוחים והאלוף ב-JSON.	Fixed NeuroEvo
בניית תשתית SharpNEAT, הגנומים נקראים ומספרי ה-ID מאותחלים מעל המקסימום.	XML מלא של האוכלוסייה והאלוף בתוך JSON.	NEAT
ה-checkpoint מועתק חזרה ל-results ו-Python מופעל עם --resume.	תיקית results של המאמן מועתקת לחרץ checkpoint יציב לפי track.	PPO/SAC

Inference

- **אבולוציה:** טעינת IBrain של האלוף למטוס יחיד והרצת היעד בלולאה בלי אבולוציה.
- **RL:** הכנת checkpoint והפעלת `mlagents-learn --resume --inference`. המדיניות קפואה, אך עדיין דרושים Python ו-Academy.

כונון בזמן ריצה

- **Hot:** שינוי תגמול או scalar תואם משתמש ב-Load -> Save מוגן, כך שהמצב הנלמד נשמר. גיבוי מונע דריסה של השמירה הידינית.
- **Cold:** שינוי גודל אוכלוסייה או ארכיטקטורת רשת אינו תואם למשקלים הישנים. שומרים הגדרות ובונים מחדש מאפס; השמירה הידינית נשארת.

אתגר מול מדיניות שמורה

המנהל הנוכחי תומך גם במרוץ Flight School בין מטוס אנושי ל-AI שמור וקפוא. הוא טוען את ההגדרות והיעד השמורים, מפעיל inference עבור ה-AI, יוצר עותק אנושי, מבטל נשק והתנגשויות הדדיות, משתמש באותו קו זינוק ומדרג לפי מספר חישוקים ולאחר מכן מהירות ממוצעת. אין למידה ואין שמירה.

הסתייגות שמירת RL	כפתור Save יכול להעתיק רק את ה-checkpoint האחרון שהמאמן כתב. התקדמות מאז אותו checkpoint אינה נשמרת, ולכן זו שמירה מוגבלת checkpoint ולא snapshot מדויק של הפריים הנוכחי.
-------------------	---

10. סיפור הדגמה נקי לבוחנים

רצף מומלץ בהדגמה

1. **בחר מסלול וסוג AI.** הסבר שהבחירה נשמרת ב־GameSession, ושהיעד בסצנה קובע את החיפוש ואת חוזה התגמול.
2. **הצג כמה מטוסים באימון.** ציין שהאוכלוסייה והפיזיקה משותפות לכל האלגוריתמים.
3. **פתח telemetry.** הסבר generation לעומת episode, max/average נוכחיים מול best היסטורי, בחירת מטוס ומגבלות visualizer ב־RL.
4. **בחר מטוס.** חבר בין ההתנהגות, התצפיות, הפעולות ופירוט התגמול.
5. **הצג save/load או inference.** הסבר את ההבדל בין מצב אימון מלא לבין replay של מדיניות אחת.
6. **אם יציב, הצג tuning או challenge.** השתמש בו כדי להסביר hot/cold או הערכת מדיניות קפואה.

נקודות תפעול שכדאי להכיר

מה לומר	נושא
Unity 6, גרסת בסיס 6000.3.10f1.	גרסת Unity
אין צעד build חיצוני; הכול רץ בתוך Unity.	ריצה אבולוציונית
נדרשת סביבת Python תואמת ML-Agents; RLParadigm יוצר YAML בכל ריצה.	ריצת RL
חיפוש המפרש מעדיף סביבה ארוזה ב־StreamingAssets, אחריה MLAGENTS_PYTHON, ואז Conda.	Windows build
ה־player חייב לרוץ עם 5004 --mlagents-port; AppBootstrap מפעיל אותו מחדש אם הפרמטר חסר.	חיבור standalone
Thin build מוריד סביבה נעולה בהרצה הראשונה; build עצמאי כולל את הסביבה שחולצה.	אריזה

לא להפריז בטענות

- אל תציג Dogfight כמצב אימון AI ממומש. קיימת תשתית משחקית, אך חסרים יעד, חיפוש קרב, צוותים, שיוך תגמול ותנאי סיום.
- אל תאמר ש־RL inference מוטמע ב־Unity. הוא מפעיל את המאמן החיצוני במצב inference.
- אל תאמר ש־Save לוכד את משקלי PPO/SAC של הרגע הנוכחי. הוא לוכד את ה־checkpoint הזמין האחרון.
- אל תאמר שלכל אלגוריתם יש IBrain או IEvolutionEngine. אלה מושגים של השיטות האבולוציוניות בתוך התהליך.

11. מגבלות ותשובות שיפור חזקות

שיפור טוב	למה זה חשוב	בעיה
starting/ready/failed וסטטוס Coroutine/state machine ב-telemetry.	המתנה לפורט יכולה לגרום ל-Editor/player להיראות קפוא.	הפעלת המאמן חוסמת
לייצא ONNX ולהריץ inference ב-Unity עם Sentis.	עדיין דרושים Python, port, Academy והסביבה הגדולה.	RL inference יקר
להציג צעד וגיל checkpoint; אפשר גם פקודת checkpoint מצד המאמן.	ה-UI עלול לרמוז על snapshot מדויק כשצעדים אחרונים חסרים.	שמירת RL מוגבלת checkpoint גדל
להוציא שירותי challenge ומעברי ריצה, ולהשאיר את המנהל ב-composition root.	אימון, inference, tuning, persistence, ו-challenge מגדילים אחריות וקושי בבדיקה.	SimulationManager גדל
להפריד התקדמות סביבה מחישוב תגמול טהור.	קידום חישוקים בתוך GetStepReward יוצר תלות נסתרת ב-inference.	לתגמול Flight School יש side effects
להזריק שירותים והפניות מפורשות ב-composition root.	static tuners, session, events ו-singletons מקשים על בדיקות מבודדות.	צימוד global/static
לרכז defaults, לאמת assets באתחול ולהוסיף migrations/tests.	ערכי Inspector גוברים על C# initializers ועלולים לסטות בשקט.	סטיית defaults סריאליים

התשובה הטובה ל"מה היית משפר ראשון?"

עדיפות מומלצת

להפוך את הפעלת מאמן ה-RL לאסינכרונית ומפורשת. זה משפר מיד את חוויית המשתמש, מונע תחושת קיפאון ויוצר גבול מצב ברור לכלל חיבור, load, telemetry, retry, ו-inference. אחר כך, ONNX/Sentis מוטמע יפשט מאוד את ההפצה.

התשובה הטובה ל"איך היית מוסיף Dogfight?"

ראשית לממש DogfightObjective וחיישן קרב, ואז להגדיר צוותים ו-spawn, בעלות על damage, תגמולי kill/death, תנאי סיום ופלט של לפחות שש פעולות. לאחר מכן להוסיף defaults, לאמת save/inference ורק אז להציג את המצב כשלם. הנשק, הבחירה והמצלמה הקיימים הם תמיכת gameplay, לא חוזה הלמידה החסר.

בדיקות שיקנו ביטחון במהירות

- שכפול settings ונתיבי שמירה לפי track.
- רוחבי חיישנים וגודל פעולות לכל צמד יעד/Al.
- Round trip של CaptureState/RestoreState ב-Classic וב-NEAT.
- סיווג hot/cold והגנת השמירה הידנית.
- התקדמות ותנאי סיום של היעדים, במיוחד מעבר חישוק ו-inference.

12. שאלות סבירות של בוחנים - תשובות קצרות

שאלה	תשובה שכדאי לפתוח בה
מהו המרכז הארכיטקטוני?	SimulationManager הוא composition root ונתב ה-tick; הוא מאציל את מחזור החיים ל-ITrainingParadigm.
למה גם Paradigm וגם Engine?	Paradigm מפשיט את מחזור האימון כולו. Engine הוא הפשטה נמוכה, אבולוציונית בלבד, ליצירת אוכלוסייה, אבולוציה, אלוף וסריאליזציה.
כיצד משוויים אלגוריתמים בצורה הוגנת?	כולם חולקים תצפיות שבחר היעד, ארבע פעולות טיסה, JetPhysics, spawn, תגמול ותנאי סיום.
למה גם JetAgent וגם JetMLAgent?	JetAgent מפעיל IBrain בתוך התהליך. JetMLAgent מתאים את אותו מטוס/פיזיקה/יעד לתקשורת עם ML-Agents ולמדיניות החיצונית.
מה קורה בכל FixedUpdate?	המנהל מנתב ל-inference, challenge או training. מחזור החיים הפעיל מתקדם, והפיזיקה ממשיכה ב-fixed timestep.
מה נשמר?	הגדרות, פרמטרי יעד, סטטיסטיקה/היסטוריה ומצב engine. אבולוציה שומרת אוכלוסייה; RL מצלם checkpoints של המאמן.
מה היכולת החסרה הגדולה ביותר?	אימון Dogfight: יש רכיבי gameplay אך אין חוזי יעד ותצפית/תגמול קרביים.
באילו patterns משתמשים?	Strategy ל-paradigms ול-engines, ממשקים ליעדים/חיישנים/מוחות, composition root במנהל ו-snapshot read ל-model UI.

רשימת בדיקה למעבר אחד

- אני יכול לתת את נאום 60 השניות בלי דף.
- אני יכול לצייר Manager -> Paradigm -> Engine/Trainer -> Environment.
- אני יכול להסביר מדוע שכבת ה-engine אינה חלה על PPO/SAC.
- אני יכול לעקוב אחר דור Fixed NeuroEvo ואחר פרק PPO/SAC.
- אני מכיר BasicFlight (12), Waypoint (19) וארבעה פלטי טיסה.
- אני יכול להסביר שמירות לפי track, מצב אבולוציוני מול checkpoint של RL, ושינויים hot/cold.
- אני מכיר את המגבלות: Dogfight לא שלם, RL חיצוני, save מוגבל checkpoint ו-inference דרך המאמן.

מילון זעיר

משמעות	מונח
משפחת יעד: Flight School, Max Altitude או Dogfight הלא-שלם.	Mode
זהות הסצנה הפעילה להגדרות, שמירות, checkpoints ו-RL run IDs.	Track
אסטרטגיית מחזור האימון העליונה: evolutionary או RL.	Paradigm
אלגוריתם אוכלוסייה אבולוציוני בלבד: Classic NeuroEvo או NEAT.	Engine
מדיניות שניתן לקרוא לה בתוך Unity; מדיניות PPO/SAC הנוכחית אינה IBrain.	Brain
המוח האבולוציוני הטוב בכל הזמנים, או metadata של פרק RL הטוב ביותר.	Champion
הרצה חוזרת של מדיניות אחת, בלי למידה ובלי שמירה חדשה.	Inference

השורה האחרונה לזכור

הפרויקט מצליח ארכיטקטונית מפני שהמטוס והמשימה משותפים, בעוד שמחזורי החיים של הלמידה נשארים שונים בצורה נכונה.